

REPOGUARDIAN AI

TEAM MEMBERS

- **Jivites Damodar**
- **Rahul LS**
- **Prajwal Priyadarshan**
- **Sukanthan**



**YOUR CODE.
OUR GUARDIAN.
ZERO REGRESSION.**

AI-powered analysis, intelligent impact detection,
and **autonomous code healing**.



ANALYZE

Deep Code
Understanding



DETECT IMPACT

Graph-based
Analysis



HEAL

Autonomous
Fixes



DELIVER SAFELY

Smart PRs with
Risk Awareness

>_ Smarter Repos. Safer Code. Stronger Projects.



```
def analyze(repo):  
    graph = build_dependency_graph(repo)  
    impact = calculate_impact(graph)  
    risks = detect_risks(impact)  
    fix = ai_suggest_fixes(risks)  
    return apply_and_validate(fix)
```

PROJECT INTRO

RepoGuardian AI is an AI-powered repository intelligence system that analyzes codebase changes, detects downstream impact, predicts risks, and generates self-healing fixes using graph-based dependency analysis and LLM-powered reasoning.

PROBLEM STATEMENT

Modern software repositories are becoming increasingly large, interconnected, and difficult to maintain.

Challenges faced by Developers

- Understanding complex code dependencies across multiple files and services
- Predicting how a small code change impacts downstream modules
- Detecting hidden architectural breakages before deployment
- Manually tracing function calls and dependency chains
- Spending excessive time debugging ripple-effect failures

Limitations of Current Tools

- Surface-level static analysis
- Limited dependency visibility
- No semantic understanding of repository structure
- No proactive impact prediction
- No autonomous debugging or self-healing capabilities
- No repository-wide contextual reasoning
- Reactive debugging instead of proactive prevention

OUR SOLUTION

Our system combines repository parsing, graph-based dependency analysis, and AI-powered reasoning to understand how code changes impact an entire project in real time.

Understands the Repository

- Performs multi-language AST parsing
- Extracts functions, classes, imports, and relationships
- Builds a structured understanding of the codebase

Tracks Dependency Impact

- Uses Neo4j graph database for dependency mapping
- Detects downstream modules affected by code changes
- Identifies hidden architectural dependencies

Predicts Risks Using AI

- Uses LLM-powered reasoning for contextual analysis
- Explains why a code change may break the system
- Calculates impact severity and risk levels

Generates Self-Healing Fixes

- Suggests backward-compatible code fixes
- Validates generated patches automatically
- Creates pull requests with AI-generated solutions

Tech Stack:

FastAPI • React • Neo4j • Pinecone • Groq Llama 3.3 • Gemini • Tree-sitter

PROJECT WORKFLOW

RepoGuardian AI — System Workflow

